

Design and Analysis of Algorithm

Lecture 21: Queue

Content



- **Introduction**
- **Operations on Queue**

Question : 4

Consider that $-$, $\times$ and $\$$ are used to indicate the operations subtraction, multiplication and exponent. Also assume that $-$ has higher precedence than $\times$, which has higher precedence than $\$$. Find the postfix notation and solve the postfix expression for the following input

- $2 \times 2 - 1 \$ 1 \$ 4 - 2$ considering all operators are $R - L$ associative

Postfix Expression: $2 \ 2 \ 1 \ - \ \times \ 1 \ 4 \ 2 \ - \ \$ \ \$$

Result: 2

Stack: Recursion

The process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

- A recursive algorithm takes one step toward solution and then recursively call itself to further move. The algorithm stops once we reach the solution.
- Since called function may further call itself, this process might continue forever. So it is essential to provide a base case to terminate this recursion process.

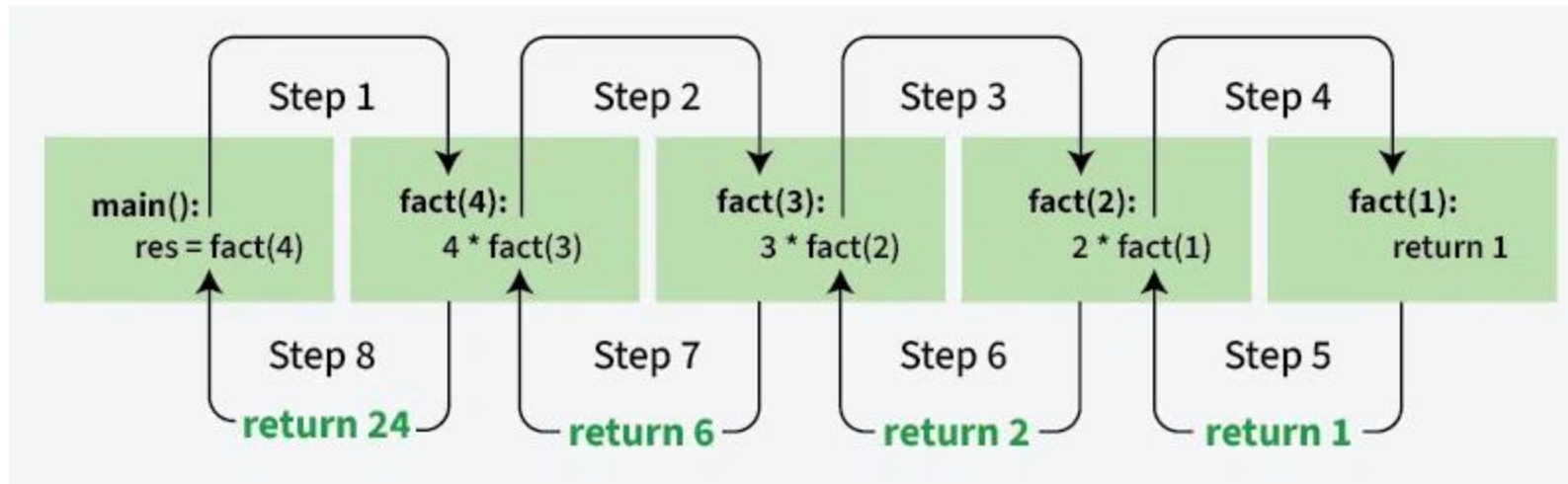
Recursion: Steps

The four major steps in the recursion are as follows

- Step1 – Define a base case
- Step2 – Define a recursive case
- Step3 – Ensure the recursion terminates
- Step4 – Combine the solutions

Recursion: Example

Factorial of a number n (where $n \geq 0$) is defined as multiplication of numbers from 1 to n . To recursive compute, we compute factorial n if we know the factorial of $(n - 1)$. The base case for factorial would be $n = 1$. We return 1 when $n = 0$.



Recursion: Stack Overflow

```
int fact(int n)
{
// wrong base case (it may cause
// stack overflow).
if (n == 100)
return 1;

else
return n*fact(n-1);
}
```

If the base case is not reached or not defined

Queue

A queue is a linear list of elements in which deletions can take place only at one end, called the front, and insertions can take place only at the other end, called the rear.

A Queue works on the FIFO process (First In First Out), i.e., the element that was inserted first will be removed first.

To implement the Queue, it is required to maintain two pointer to the *front* and *rear* of the queue.

Queue: Basic Idea

Queue is an abstract data structure, somewhat similar to Stacks. Unlike stacks, a queue is open at both its ends. One end is always used to insert data (enqueue) and the other is used to remove data (dequeue).



Queue: Application

Job Scheduling

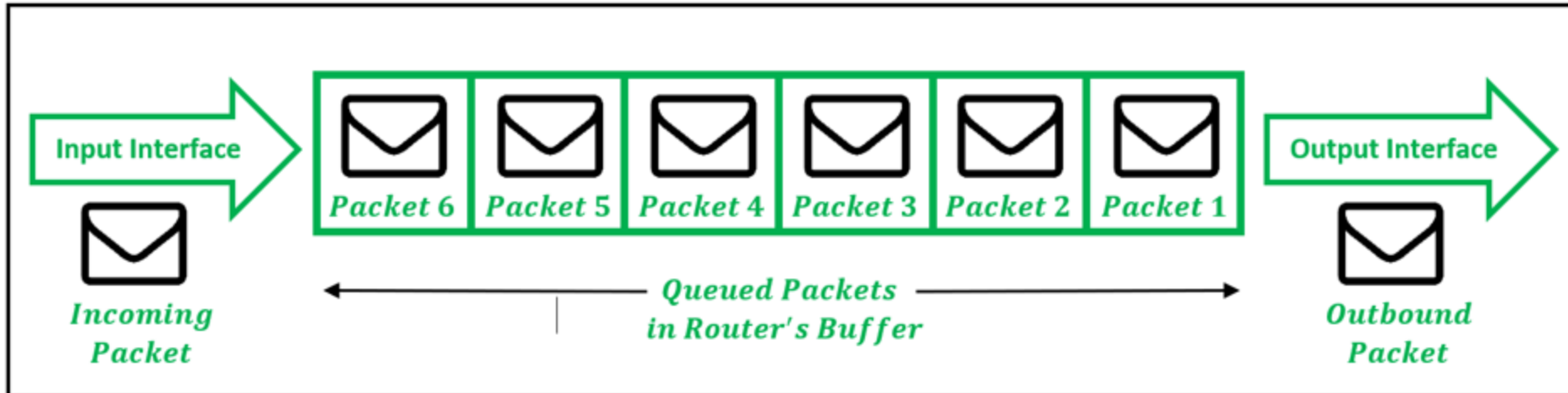
Queues are used to manage the execution order of tasks, ensuring efficient CPU utilization and managing system workload by organizing processes in a first-in, first-out (FIFO) manner



Queue: Application

Network Routing

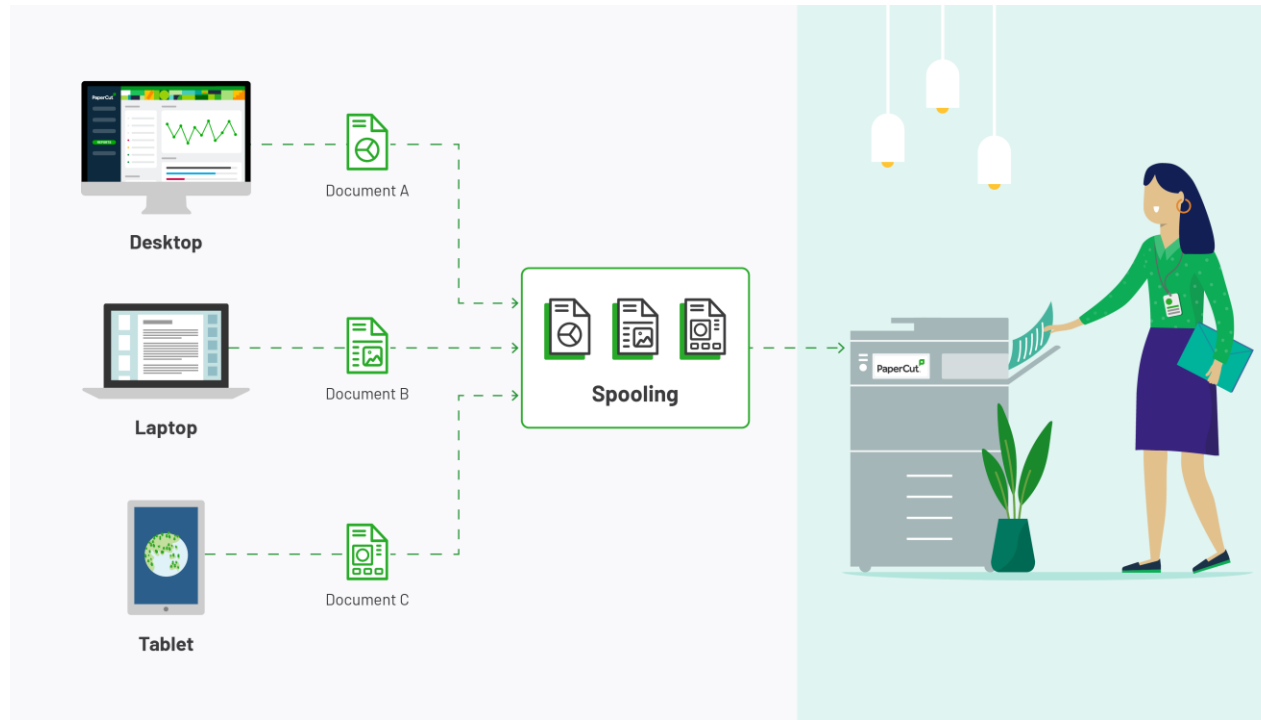
Routers use queues to store packets of data that are waiting to be transmitted, ensuring efficient and orderly network traffic flow.



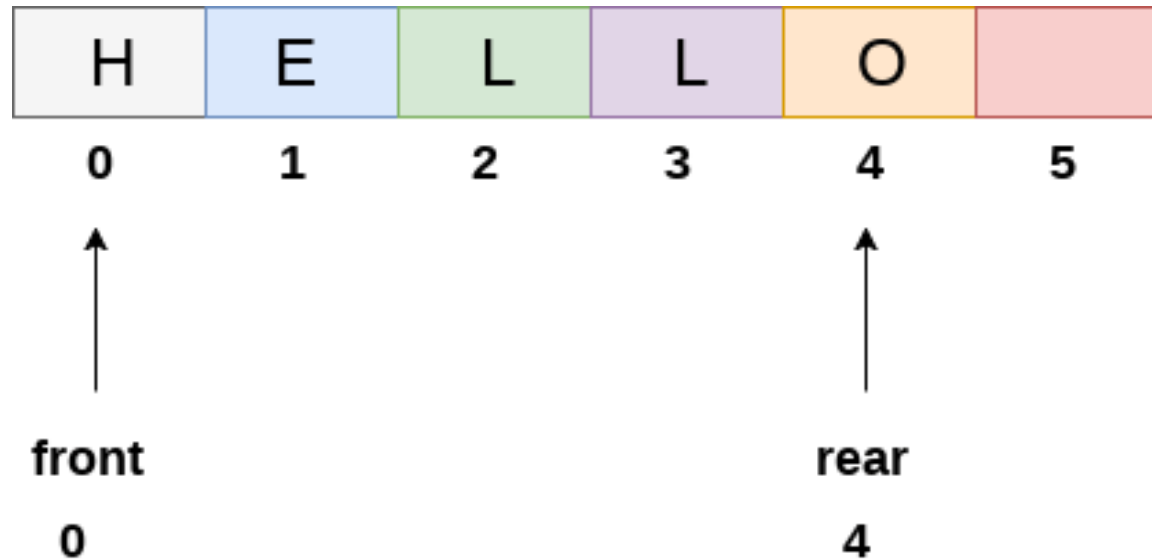
Stack: Application

Printer Spooling:

When multiple print jobs are submitted to a printer, they are stored in a queue until the printer is ready to process them, allowing users to continue working while printing.



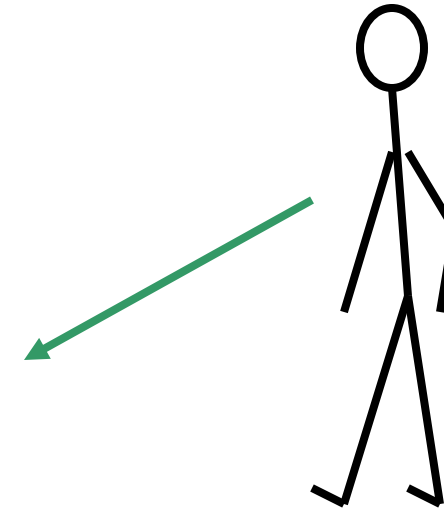
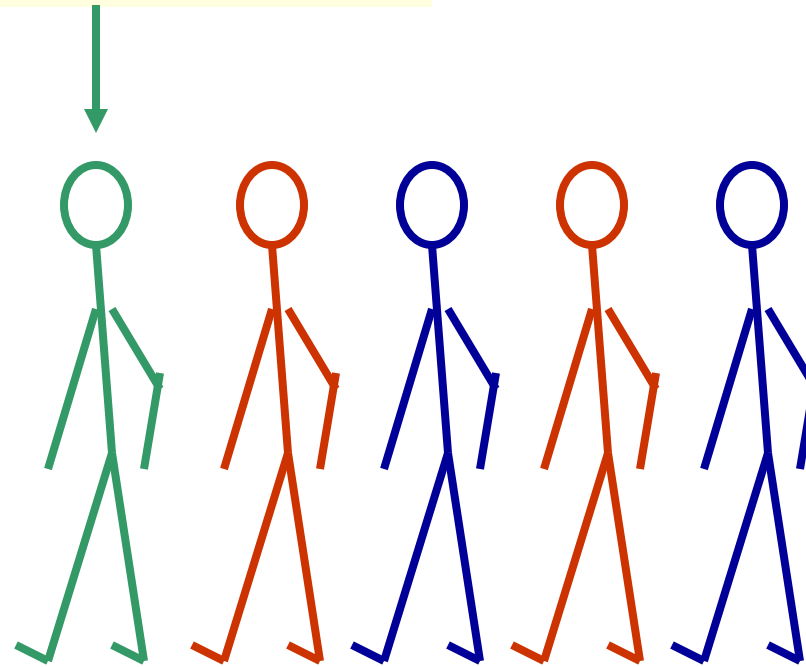
Queue: Representation



Queue: Conceptual View

Enqueue

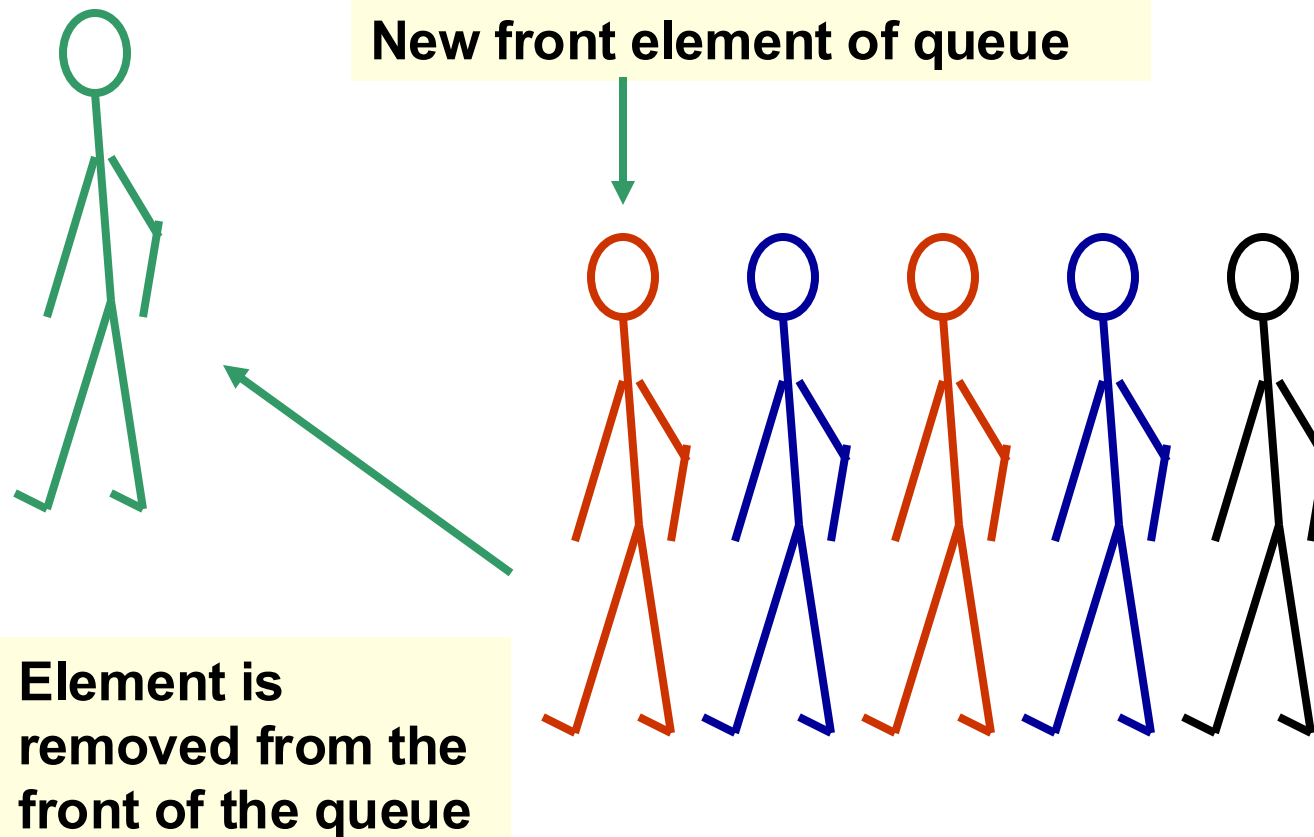
Front of queue



New element is
added to the rear
of the queue

Queue: Conceptual View

Dequeue



Queue: Operations

The primary operations on a Queue, a First-In, First-Out (LIFO) data structure, are

Enqueue:

- Adds a new element to queue.

Dequeue:

- Removes the element from the queue.

Peek:

- Returns the element available at the front of the queue without deleting it

isEmpty:

- Checks if the queue is empty.

isFull:

- Checks if the queue is full

size:

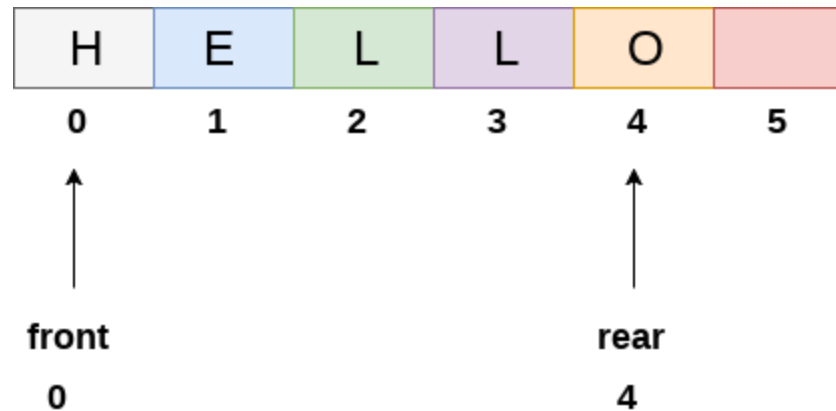
- Returns the number of elements in the queue.

Rear:

- Returns the element available at the rear of the queue without deleting it

Queue: Array Representation

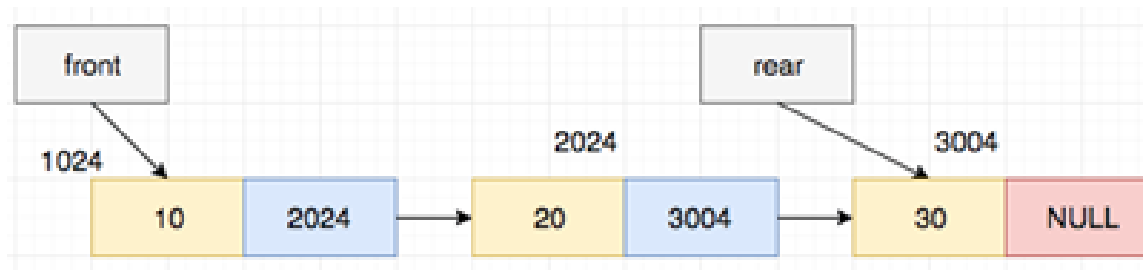
Queue can easily be implemented using array. There are two variables i.e. *front* and *rear*, that are implemented in the case of every queue. Front and rear variables point to the position from where insertions and deletions are performed in a queue.



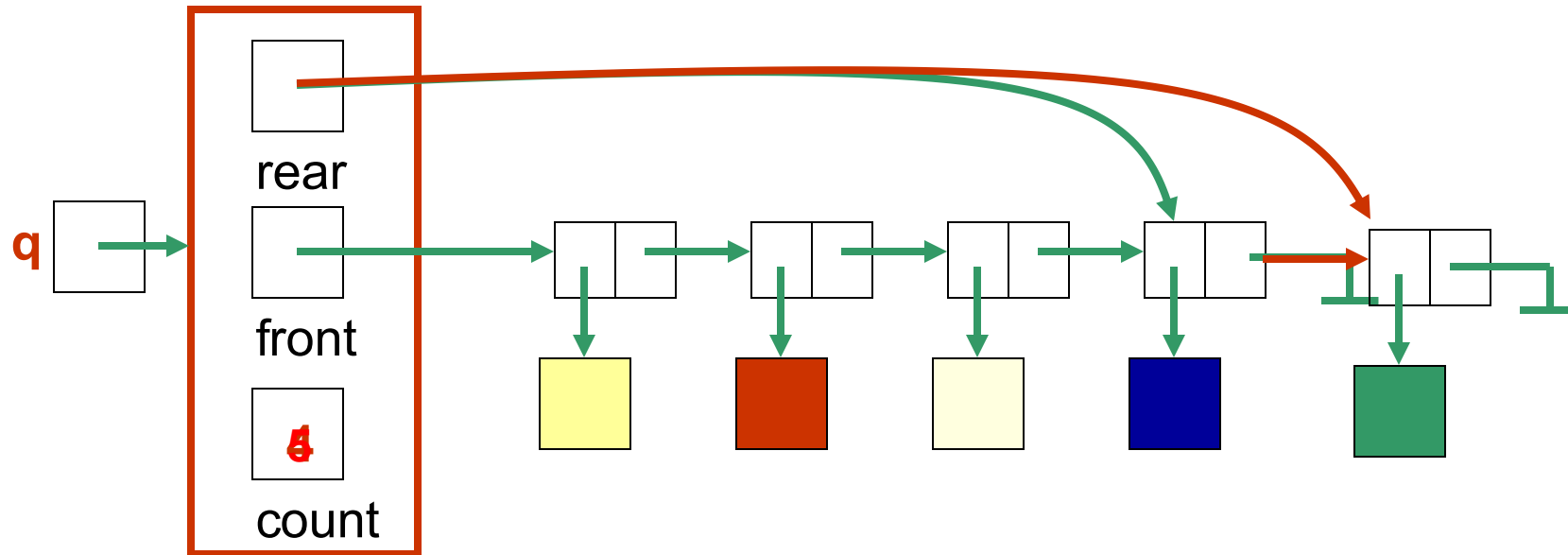
Queue

Queue: Linked List Representation

- A queue can be represented as a *linked list of nodes*, with each node containing a data item
- We need *two* pointers for the linked list
 - A pointer to the beginning of the linked list (*front* of queue)
 - A pointer to the end of the linked list (*rear* of queue)
- We will also have a *count* of the number of items in the queue



Queue: Enqueue



Queue: Dequeue

